

```
=====
                        BINARY SEARCH PROGRAM
                        in C Language
=====
```

```
PROGRAM DESCRIPTION:
```

This program implements the Binary Search algorithm to find a specific element in a sorted array. Binary search is an efficient searching technique with a time complexity of $O(\log n)$.

```
=====
                        SOURCE CODE
=====
```

```
#include <stdio.h>

int main() {
    int arr[100], n, key;
    int low, high, mid;
    int found = 0;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements in sorted order:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter the element to search: ");
    scanf("%d", &key);

    low = 0;
    high = n - 1;

    while (low <= high) {
        mid = (low + high) / 2;

        if (arr[mid] == key) {
            printf("Element found at position %d\n", mid + 1);
            found = 1;
            break;
        }
        else if (arr[mid] < key) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }

    if (!found) {
        printf("Element not found\n");
    }

    return 0;
}
```

```
=====
                        PROGRAM EXPLANATION
=====
```

1. VARIABLE DECLARATIONS:

- arr[100]: Array to store elements (max 100 elements)

- n: Number of elements
- key: Element to search for
- low: Starting index
- high: Ending index
- mid: Middle index
- found: Flag to track if element is found

2. INPUT PHASE:

- Takes number of elements from user
- Accepts sorted array elements
- Accepts the element to search

3. BINARY SEARCH ALGORITHM:

- Initializes low = 0, high = n-1
- Repeatedly divides search space in half:
 - * Calculates mid = (low + high) / 2
 - * Compares arr[mid] with key
 - * If equal: Element found, break
 - * If arr[mid] < key: Search right half (low = mid + 1)
 - * If arr[mid] > key: Search left half (high = mid - 1)
- Continues until element is found or search space is exhausted

4. OUTPUT:

- Displays position if element is found
- Displays message if element is not found

HOW TO COMPILE & RUN

Compilation:

gcc -o binary_search binary_search.c

Execution:

./binary_search

Example Input:

Enter the number of elements: 5
 Enter 5 elements in sorted order:
 10 20 30 40 50
 Enter the element to search: 30

Example Output:

Element found at position 3

KEY FEATURES

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Precondition: Array must be sorted in ascending order

Advantages:

- Much faster than linear search for large datasets
- Efficient $O(\log n)$ time complexity
- Suitable for sorted data
- Optimal for searching in large datasets

Limitations:

- Array must be pre-sorted
- Not suitable for unsorted data
- Takes more time for very small arrays (overhead)
- Requires random access to elements

ALGORITHM STEPS

Step 1: Initialize low = 0, high = n-1
Step 2: While low <= high:
 a) Calculate mid = (low + high) / 2
 b) If arr[mid] == key, return mid (element found)
 c) If arr[mid] < key, set low = mid + 1 (search right)
 d) If arr[mid] > key, set high = mid - 1 (search left)
Step 3: If loop ends without finding, element is not in array

EXAMPLE WALKTHROUGH

Array: [10, 20, 30, 40, 50]
Search for: 30

Iteration 1:
 low = 0, high = 4
 mid = (0 + 4) / 2 = 2
 arr[2] = 30
 30 == 30 â†’ FOUND at position 3 (mid + 1)

Iteration 1:
 low = 0, high = 4
 mid = (0 + 4) / 2 = 2
 arr[2] = 30
 30 < 40 â†’ Search in left half

Iteration 2:
 low = 0, high = 1
 mid = (0 + 1) / 2 = 0
 arr[0] = 10
 10 < 40 â†’ Search in right half

Iteration 3:
 low = 1, high = 1
 mid = (1 + 1) / 2 = 1
 arr[1] = 20
 20 < 40 â†’ Search in right half

Iteration 4:
 low = 2, high = 1
 low > high â†’ LOOP ENDS
 Element not found

COMPARISON WITH LINEAR SEARCH

	Binary Search	Linear Search
Time Complexity:	$O(\log n)$	$O(n)$
Space Complexity:	$O(1)$	$O(1)$
Prerequisite:	Sorted array	Any array
Best Case:	$O(1)$	$O(1)$
Worst Case:	$O(\log n)$	$O(n)$
Average Case:	$O(\log n)$	$O(n/2)$

For a dataset of 1,000,000 elements:
- Binary Search: ~20 comparisons
- Linear Search: ~500,000 comparisons (average)

=====